

DML operation

TCL ---- transaction control language

commit	It makes the changes permanent
rollback	It undo the changes
savepoint	It puts some marker

to turn off autocommit

set autocommit=0

to turn on autocommit

set autocommit=1

if autocommit is off

10 records are there in mytable

drop table dept; -----this is autocommit

rollback;

if we have mytable which has 5 records

commit;

insert -3

insert-4

savepoint A

delete 1

update 1

Savepoint B

insert-3

insert 2

Savepoint c

delete 1

rollback to C

DCL ----Data control language

grant	It is used for assigning the permission
revoke	It is used for revoking the permission

to assign permission to all database, all table, all privileges

```
GRANT ALL PRIVILEGES ON * . * TO 'newuser'@'localhost';
```

to assign insertType equation here., create, select permissions on test database emp table to newuser

```
Grant select,create,insert on test.emp to 'newuser'@'localhost'
```

to make the changed permission permanent

```
FLUSH PRIVILEGES;
```

How To Grant Different User Permissions

Here is a short list of other common possible permissions that users can enjoy.

- ALL PRIVILEGES- as we saw previously, this would allow a MySQL user full access to a designated database (or if no database is selected, global access across the system)
- CREATE- allows them to create new tables or databases
- DROP- allows them to them to delete tables or databases
- DELETE- allows them to delete rows from tables
- INSERT- allows them to insert rows into tables
- SELECT- allows them to use the SELECT command to read through databases
- UPDATE- allow them to update table rows
- GRANT OPTION- allows them to grant or remove other users' privileges

To provide a specific user with a permission, you can use this framework:

- GRANT type_of_permission ON database_name.table_name TO 'username'@'localhost';
- Grant select,insert on 'mydb.dept' to 'user2'@'localhost'
- Grant select,insert on 'mydb.dept' to 'user3'@'localhost' with grant option

If you need to revoke a permission, the structure is almost identical to granting it:

- REVOKE type_of_permission ON database_name.table_name FROM 'username'@'localhost';

```
Revoke insert on test.dept from 'username'@'localhost';
```

Note that when revoking permissions, the syntax requires that you use FROM, instead of TO as we used when granting permissions.

You can review a user's current permissions by running the following:

- SHOW GRANTS FOR 'username'@'localhost';

PLSQL (Procedural Language Structured Query Language)

Why we use PL SQL

1. we can hide table names from the developer of the middleware application, which increases the security of the database.
2. For a particular task, if we need to execute many queries, then we may wrap these queries in a procedure, and call the procedure from middleware application, once, execute all the queries, complete the task and go back, this will reduce the network traffic, also improves performance efficiency of the middleware application. so it reduces the interaction between middleware program and database.
3. If any of the query is complex, then we may hide the query inside the procedure
4. Procedures will also reduce the network traffic.

in plsql we can write 3 types of blocks

procedure	If you want to write business logic and do not want to use return statement, then use procedure
function	If you want to return single value as output, then use functions
triggers	if you want to write procedures which gets executed automatically, is called as triggers

in procedure we can pass 3 types of parameters

in	these types of parameters are used for passing the value as i/p these are readonly parameters, its value cannot be changed inside the procedure this is default type of parameter
out	these types of parameters are used for getting the output these are writeonly parameters, its value can be assigned or changed inside the procedure
inout	these types of parameters are used for passing input as well as getting the output these are read and write parameters, using these parameters we may pass the value and we may change its value

While writing procedures or function we need to change the delimiter

delimiter //

1. to write procedure to insert record in department table

delimiter //

create procedure insertdept(in dno int,in dnm varchar(20),

in dloc varchar(20))

begin

insert into dept values(dno,dnm,dloc);

end//

delimiter ;

to check whether procedure works correctly or not

mysql> call insertdept(200,'admin','pune');

2. to get number of employees in each department and minsal maxsal

delimiter //

create procedure getdata(in dno int,out cnt int,

out minsal float(9,2), out maxsal float(9,2))

begin

select count(*),min(sal),max(sal) into cnt,minsal,maxsal

from emp

where deptno=dno;

#to print all values

-- this is comment

select cnt,minsal,maxsal;

end//

delimiter ;

to check whether procedure works correctly or not

mysql> call getdata (10,@cnt,@min,@max);

3. increment cnt by given val

delimiter //

create procedure incrementcnt(inout cnt int,in val int)

begin

set cnt=cnt+val;

```
end//
```

```
delimiter ;
```

to check whether procedure works correctly or not

```
mysql> set @c=5
```

```
call incrementcnt(@c,12)
```

```
select @c;
```

4. write a procedure to find sal+comm of a employee whose empno is given

```
delimiter //
```

```
create procedure getadetails(in eno int,out netsal float(9,2))
```

```
begin
```

```
    select sal+ifnull(comm,0) into netsal
```

```
    from emp
```

```
    where empno=eno;
```

```
end//
```

```
delimiter ;
```

to check whether procedure works correctly or not

```
call getadetails(7902,@s)
```

```
select @s;
```

in above example, select ... into statement can be used only inside pl sql blocks, the select query should return single row as output. number of column names before into and number of variables after into should be same.

@s is session variables. these variables will remain available till the time you logout.

Syntax for if—else

IF expression THEN statements; ELSE else-statements; END IF;	Using If ----else----- IF expression THEN statements; ELSEIF elseif-expression THEN elseif-statements; ... ELSE
--	---

	else-statements; END IF;
--	---------------------------------

if comm is null or 0 "poor performance"
 comm <=300 "ok performance"
 comm >300 and <= 500 "good performance"
 otherwise "excellent performance"

```

delimiter //
create procedure getfeedback(in eno int,out response varchar(40))
begin
  declare vcomm float(9,2);

  select comm into vcomm
  from emp
  where empno=eno;

  select vcomm;
  if vcomm is null or vcomm=0 then
    set response='poor performance';
  elseif vcomm<=300 then
    set response='ok performance';
  elseif vcomm<=500 then
    set response='good performance';
  else
    set response='excellent performance';
  end if;
  select response;
end//
delimiter ;
call getfeedback(7902,@r);
select @r;

```

loops in plsql

while expression do statements; end while;	It is a top tested loop, statements inside loops will get executed till the condition is true.
repeat statements until expression end repeat;	It is a bottom tested loop, and statements inside this loop will get executed until the condition is false.
label1: loop if condition then leave label1 end if; end loop;	This is infinite loop, and will continue execution till leave statement gets executed, leave statement is same as break statement, it stops the loop forcefully

	In the loop you may use iterate statement, it is similar to continue statement. It will transfer the control at the beginning of the loop
--	--

example while loop

delimiter //

create procedure displaywhile(in num int)

begin

declare str varchar(200) default '';

declare n int default 1;

while n<=num do

set str=concat(str,n,','); #1,2,3,4,5,

set n=n+1;

end while;

#remove last , from str

set str=left(str,length(str)-1);

select str;

end//

delimiter ;

loop-endloop example

delimiter //

create procedure loopdemo(in num int)

begin

declare cnt int default 1;

declare str varchar(100) default '';

label1:loop

set str=concat(str,cnt,',');

set cnt=cnt+1;

if cnt>num then

leave label1;

end if;

end loop;

set str=left(str,length(str)-1);

select str;

end//

delimiter ;

In mysql the select statement which returns multiple rows is allowed to be written inside the procedure, but in other databases like oracle, allows only select....into statement inside the procedure

hence if you need multiple rows from a table inside procedure, then we use cursor.

step by step procedure to use cursor

1. declare the cursor
2. declare continue handler to stop the loop

3. open cursor -> the data will be populated in the cursor
4. fetch the next row in the cursor
5. check if it is last row, then stop the loop and goto step 8
6. process the row
7. repeat steps 4 to 6 till the data is available in the cursor
8. close cursor

```

delimiter //
create procedure displayallemp()
begin
    declare vname,vjob varchar(20);
        declare v_finished,vempno int default 0;
        declare vsal float(9,2);
    declare empcur cursor for select empno,ename,job,sal from emp;
    declare continue handler for NOT FOUND set v_finished=1;
    open empcur;
label1: loop
    fetch empcur into vempno,vname,vjob,vsal;
        if v_finished=1 then
            leave label1;
        end if;
        select vempno,vname,vjob,vsal;
    end loop;
    close empcur;

end//
delimiter ;

```

```

create procedure updateempsal()
-> begin
-> declare vname,vjob varchar(20);
-> declare v_finished,vempno int default 0;
-> declare vsal,vnewsal float(9,2);
-> declare empcur cursor for select empno,ename,job,sal from emp;
-> declare continue handler for NOT FOUND set v_finished=1;
-> open empcur;
-> label1: loop
->     fetch empcur into vempno,vname,vjob,vsal;
->     if v_finished=1 then
-> leave label1;
->     end if;
->     #select vempno,vname,vjob,vsal;
->     if vjob='Manager' then
->         set vnewsal=vsal*1.10;
->
->     elseif vjob='Analyst' then
->         set vnewsal=vsal*1.12;

```

```
->
-> elseif vjob='Clerk' then
->   set vnewsal=vsal*1.15;
->
-> else
->   set vnewsal=vsal*1.08;
->
->   end if;
-> update emp
-> set sal=vnewsal
-> where empno=vempno;
-> select vempno,vname,vsal,vjob,vnewsal;
-> end loop;
-> close empcur;
-> end//
```